

This document is intended to guide stakeholders through the setup of the Slack bot for the Automated Incident Management System. It focuses exclusively on the configuration required within the Slack workspace and outlines the prerequisites necessary for successful integration with the backend application.

Please note that the Slack user interface is subject to change. The screenshots included in this document reflect the Slack UI as of January 20, 2026, and may differ from future versions.

Slack Bot Setup

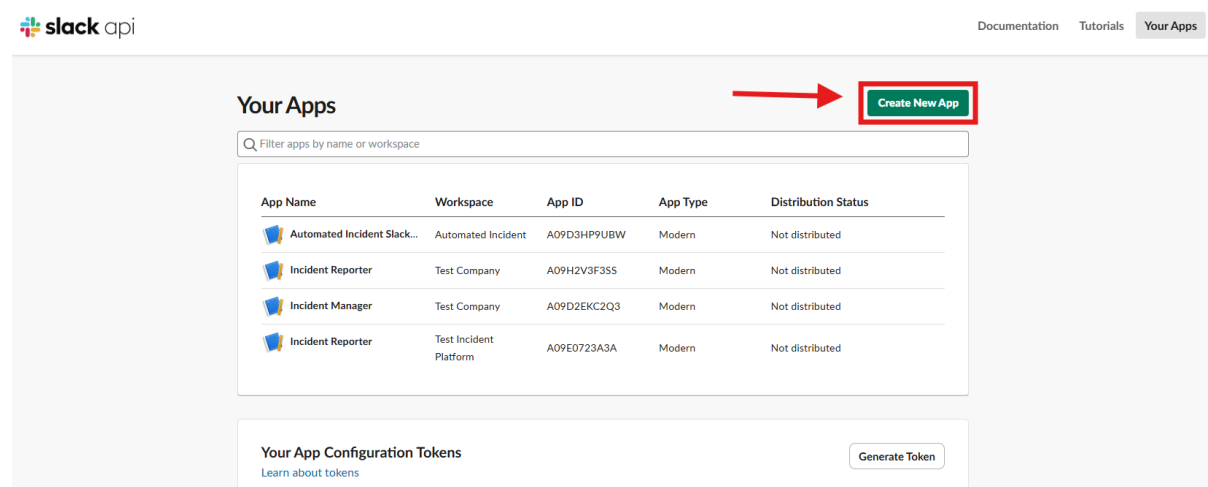
The application uses Slack Bots as an integral part to communicate between the main backend, the developer on Slack, and to the original reporter (if sent from Slack). The Slack bot requires a setup before you can run the backend. There are two types of bots that are used:

1. Reporter Bot – Manages all actions and communications between the backend and the reporter on Slack
2. Manager Bot – Manages all actions and communications between the backend and the developer on Slack

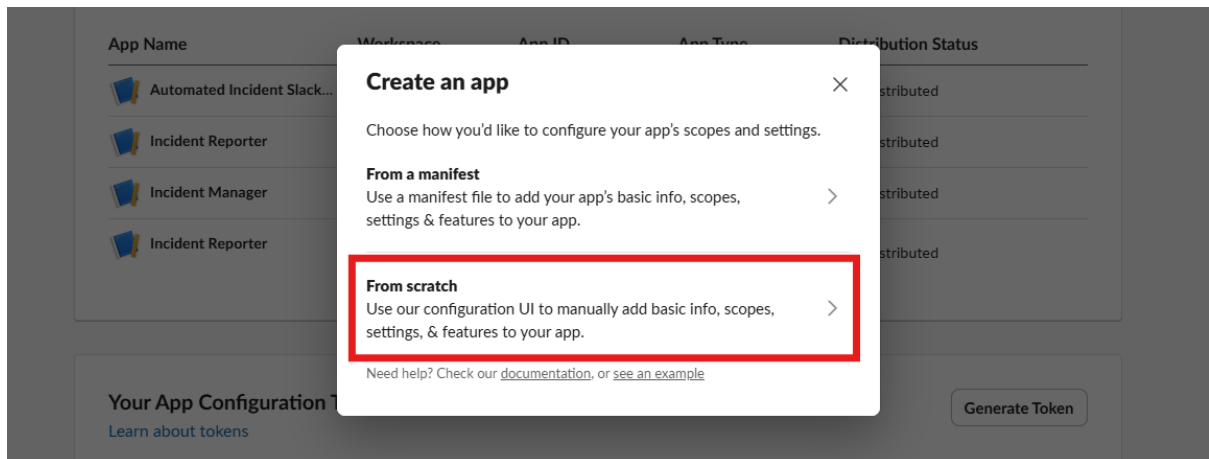
We will create two Slack bot apps, one for “Reporter” and one for “Manager”. We’ll start by showing you how to setup “Reporter”.

Reporter Bot

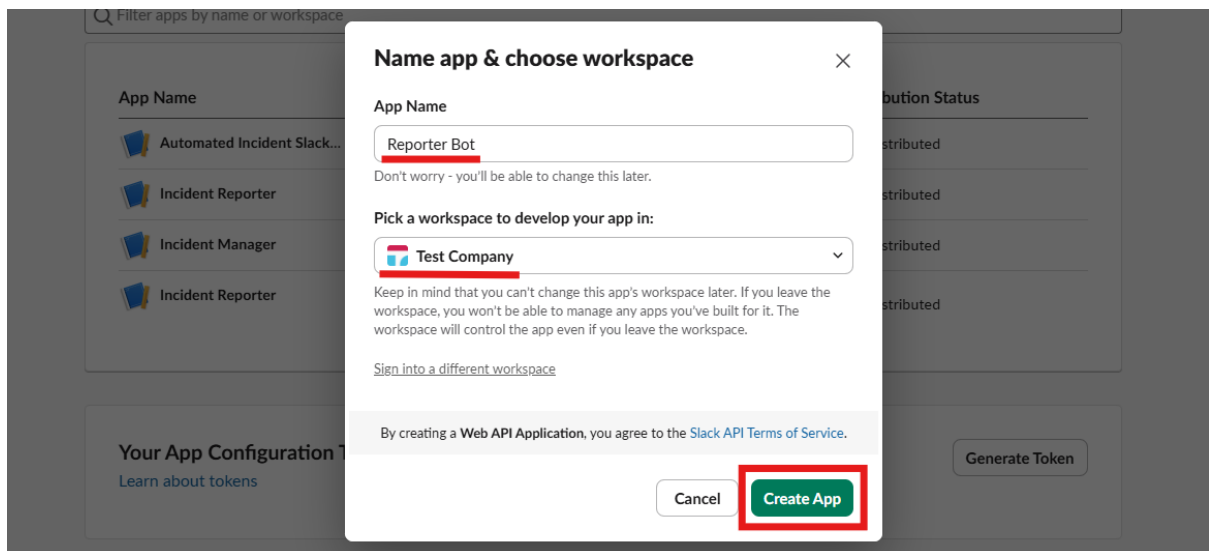
To begin, it’s essential that you already have a Slack workspace setup. The next step is easy. Visit api.slack.com/apps and click on the “Create New App” button on the top right.



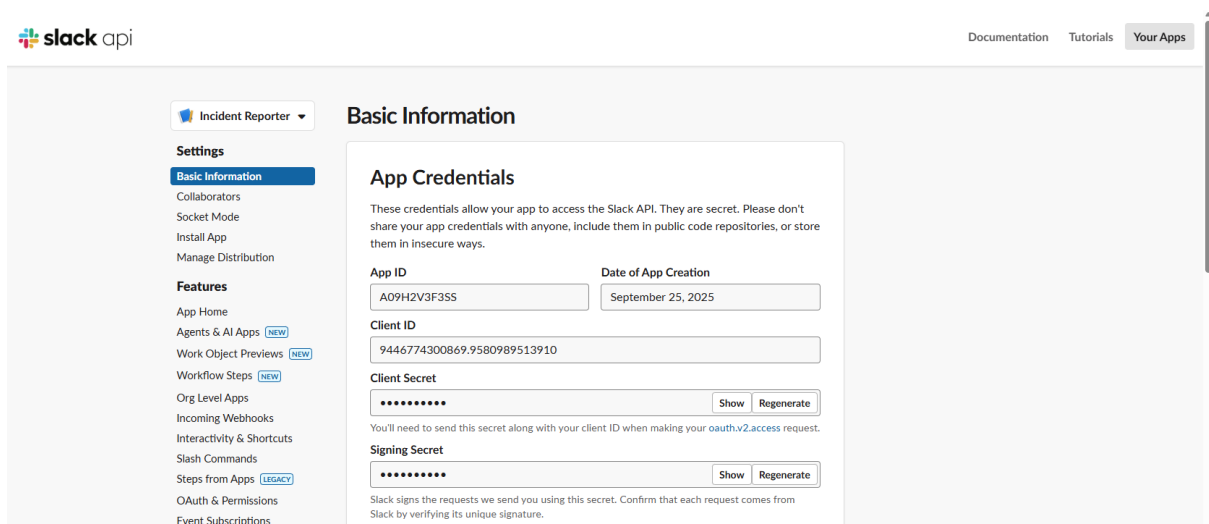
Next, a modal view will pop up asking if you would like to create an app from a manifest or from scratch. Select “From Scratch”.



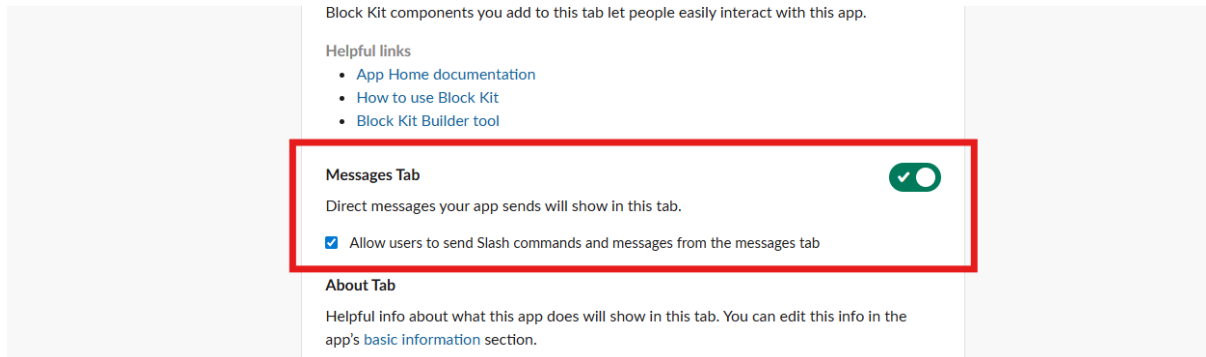
Next screen will prompt for the bot's name and the workspace to install it in. Fill it according to your needs.



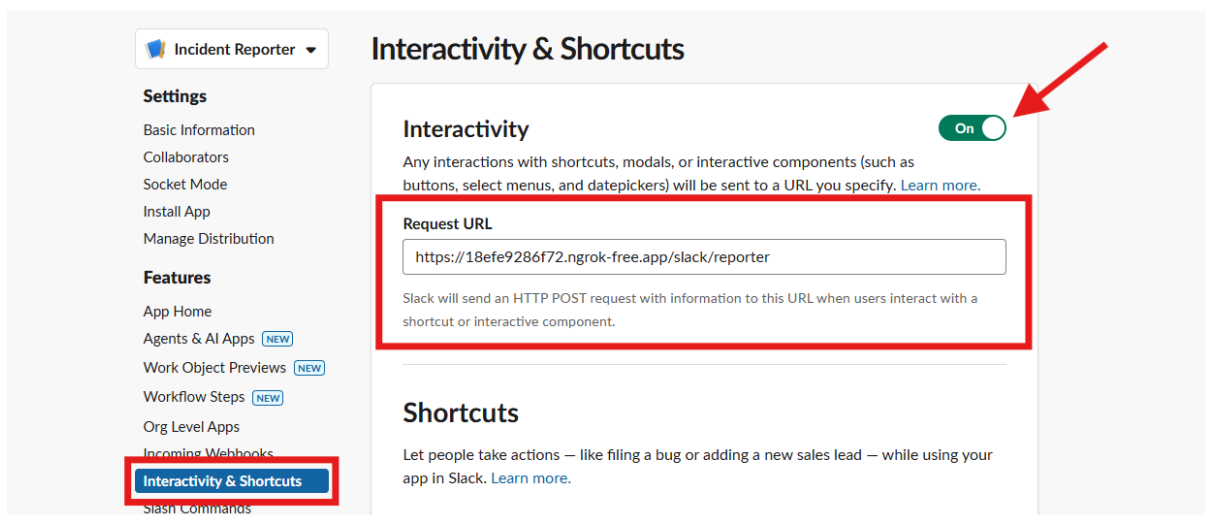
Next up, you'll see a window like below. There are many panels in the Slack bot setup dashboard, and we'll go through which one is needed. We will first setup the permissions that are needed for the "Reporter" bot to function.



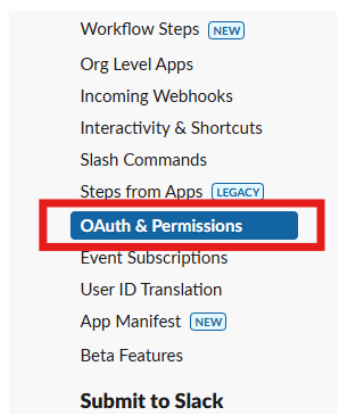
On the left side of the page, we see sections of links, some are under “Settings” and some are under “Features”, among others. Click on the “App Home” tab and scroll down till you find “Messages Tab”. Make sure this option is enabled. This will allow the bot to be exposed to users of the Slack workspace via private message.



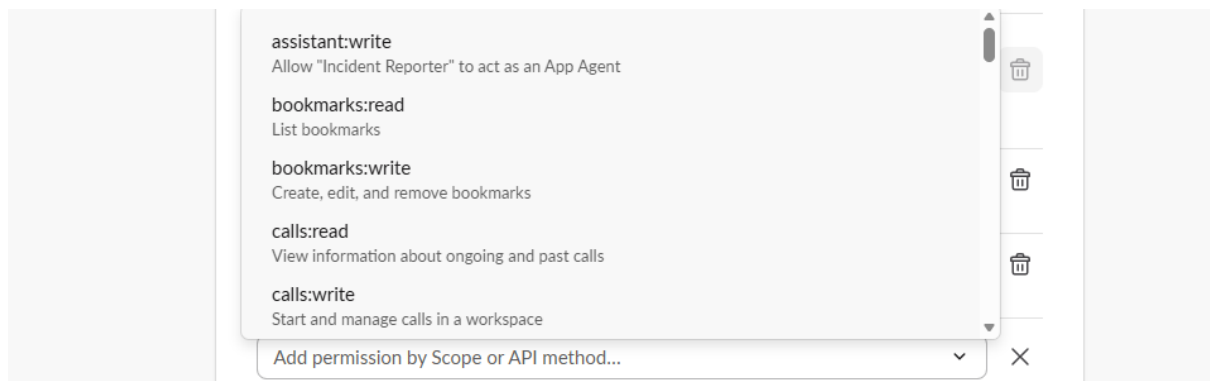
Next, look at the left side panel again and click on the “Interactivity & Shortcuts” tab. Enable the option for “Interactivity”, and paste the URL that corresponds to your backend, followed by “/slack/reporter” as this is the exact endpoint for the reporter bot in the application. In the example below, we have deployed it via Ngrok.



Next, go to the panel again and click on “OAuth & Permissions”. This is where we enable permissions for the Slack bot.

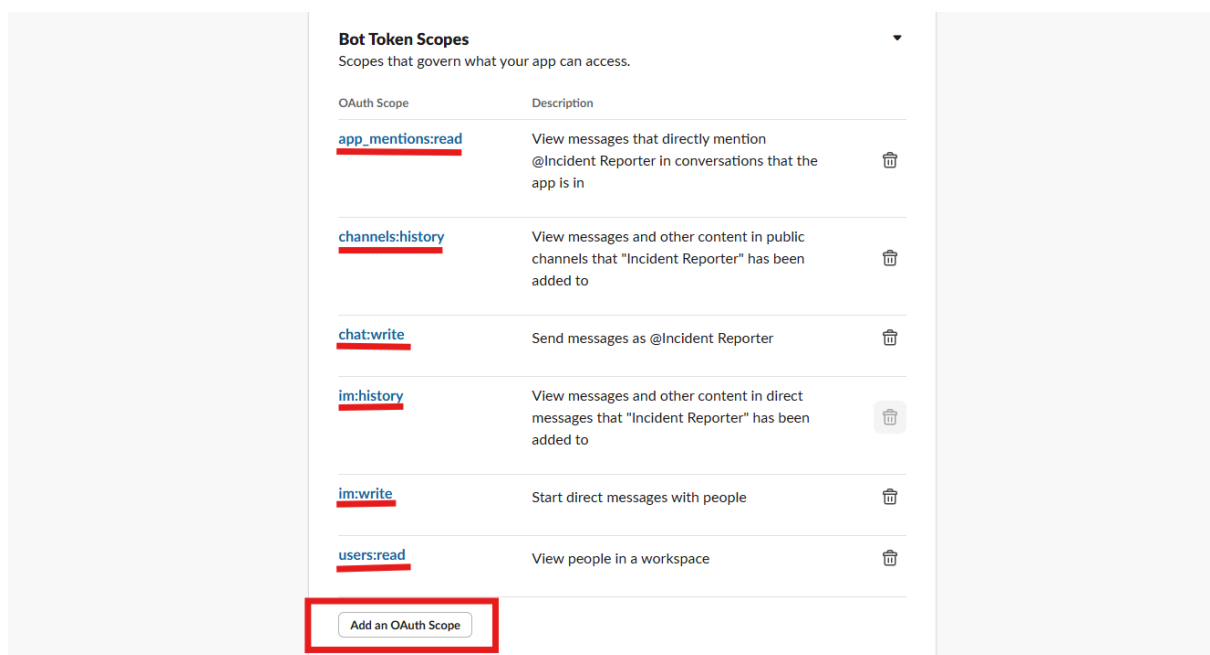


Scroll down the page and find the “Scopes” section. Under “Bot Token Scopes”, we see a button that says, “Add an OAuth Scope”. Click this button to reveal an autocomplete with the list of permissions that are available.

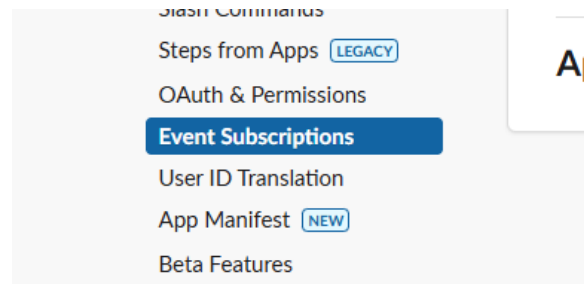


For our reporter bot, we will need the following token scopes:

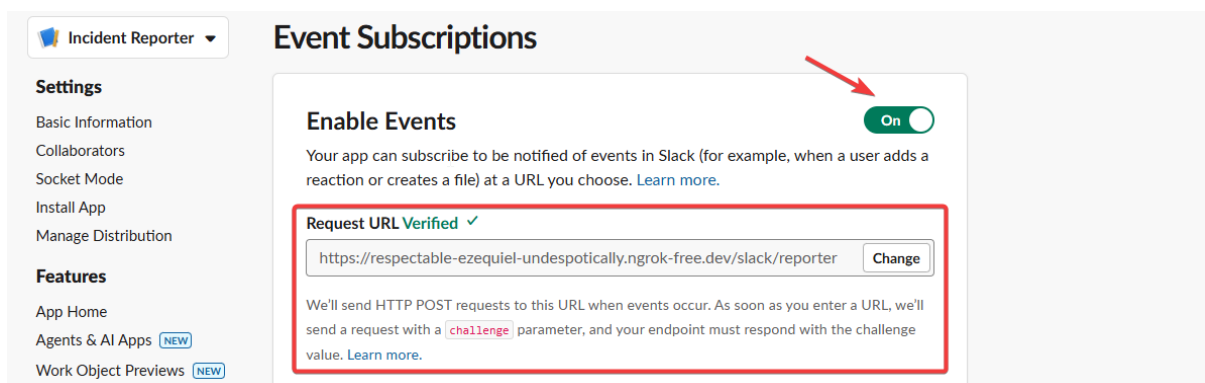
<code>app_mentions:read</code>	<code>im:history</code>
<code>channels:history</code>	<code>im:write</code>
<code>chat:write</code>	<code>users:read</code>



These permissions ensure that our reporter bot has the ability to read messages from private messages, send messages back, read user profiles, and much more. Next, the last thing that we should enable is the “Event Subscriptions”. Find this again in the left side panel.



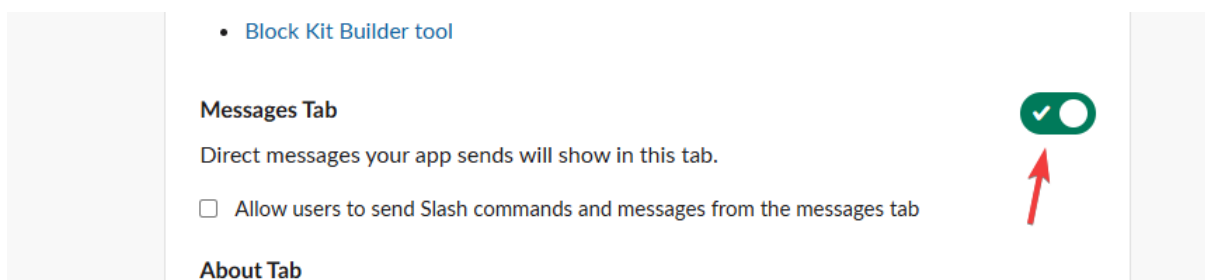
This time, another request URL appears. This is where our bot can be notified of events/actions. Pass in your backend deployment URL, followed by “/slack/reporter”. This is the main endpoint for the reporter bot defined in the backend app.



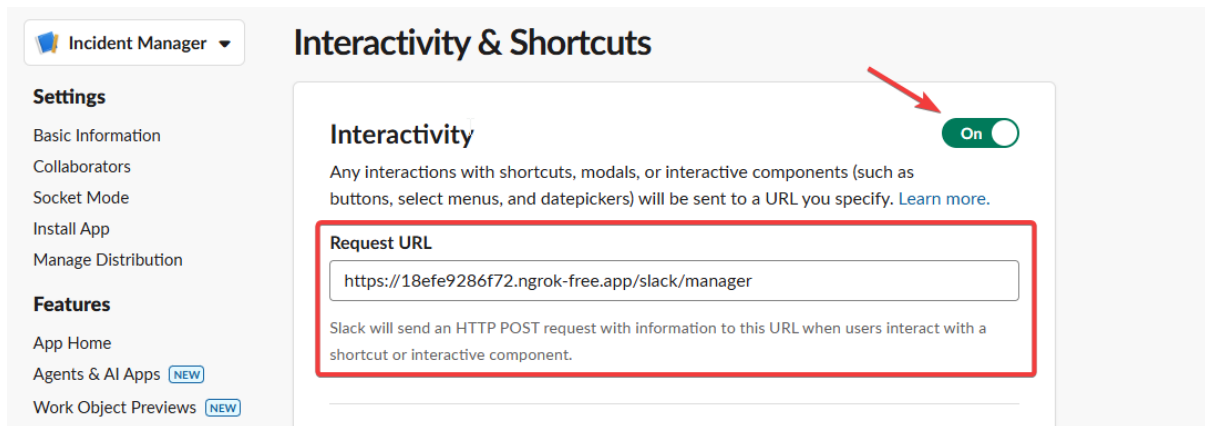
Manager Bot

We are now done setting up the reporter bot. We shall now move on to creating our second bot, the manager bot, which is in charge of communications between backend and developers.

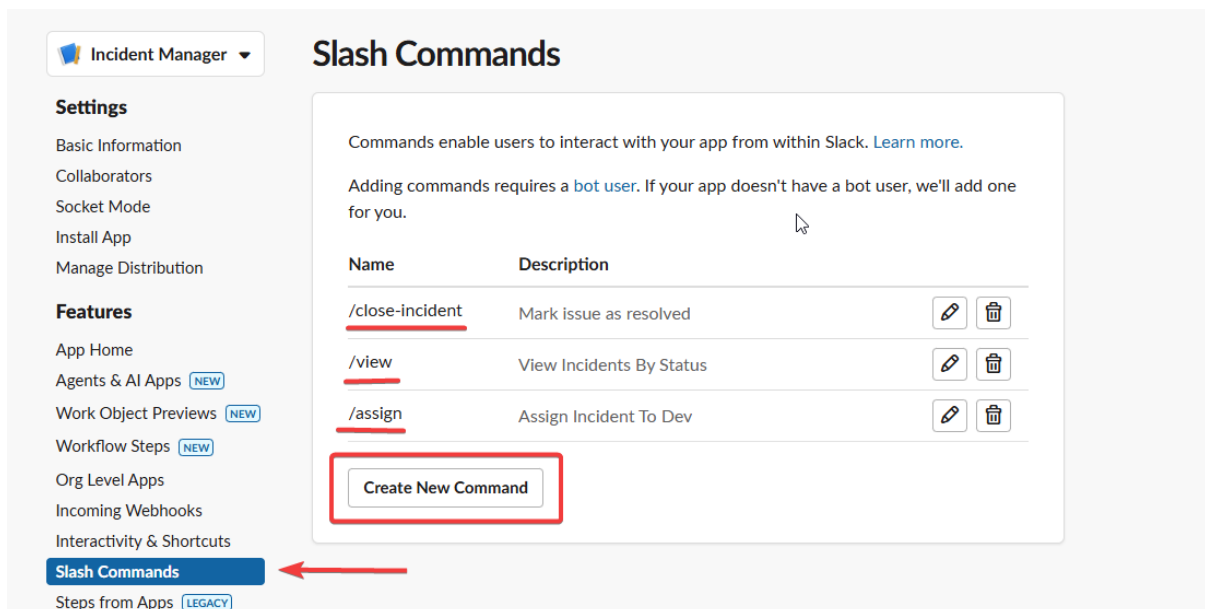
Follow the steps from the beginning to create a new app from scratch. Then visit the “App Home” once more to make the “Messages Tab” settings as enabled.



Next, go to the “Interactivity & Shortcuts” tab again and add in the request URL of the deployed backend, followed by a “/slack/manager”.



Next, this is something that we did not do with the reporter bot: slash commands privileges inside the Slack workspace. Slash commands will allow workspace admins to see all incident details and manage them. This is how we enable it. First, head over to the “Slash Commands” section in the left side panel.



In the image above we have three slash command configurations:

1. `/close-incident` – For developers to mark an incident issue as resolved and notify the original reporter
2. `/view` – For admins to view all incidents by statuses of the incidents (open, resolved, assigned, dismissed)
3. `/assign` – For admins to assign a developer to an incident

Click on the “Create New Command” button on this page. We’re going to create the “`/close-incident`” endpoint.

Close Incident

Edit Command

`{{DEPLOYMENT_URL}}/slack/manager/close_incident`

Command	/close-incident
Request URL	https://respectable-ezequiel-undesp...
Short Description	Mark issue as resolved
Usage Hint	[reason (required)]

Optionally list any parameters that can be passed.

A modal should appear asking for command details. Fill in the input boxes for “Command”, “Request URL”, “Short Description”, and “Usage Hint”. In the image above we have emphasized this.

The “Request URL” input should be filled with your backend deployment URL, followed by “/slack/manager/close_incident”. This is the endpoint corresponding to the endpoint for “Close Incident” command in the backend. When you’re done, click on the “Save” button below.

View Incidents

Click on the “Create New Command” button command again and fill in the input boxes for our “View” command, as per below.

Edit Command

`{{DEPLOYMENT_URL}}/slack/manager/view`

Command	/view
Request URL	https://kingfish-flexible-monster.ngr...
Short Description	View Incidents By Status
Usage Hint	[open, assigned, resolved, dismissed]

Optionally list any parameters that can be passed.

Assign Incident

Then after this hit save, and we shall do one more slash command, the “Assign” command. Click on the create button once more and fill in the inputs as per below.

Edit Command

`{{DEPLOYMENT_URL}}/slack/manager/assign`

Command	/assign
Request URL	https://kingfish-flexible-monster.ngr...
Short Description	Assign Incident To Dev
Usage Hint	<developer-id> <incident-id>

Optionally list any parameters that can be passed.

We are all done with creating slash commands for the manager bot. Next, we have to set permissions for manager. Visit “OAuth & Permissions”.

Bot Token Scopes
Scopes that govern what your app can access.

OAuth Scope	Description
channels:manage	Manage public channels that "Incident Manager" has been added to and create new ones
channels:read	View basic information about public channels in a workspace
channels:write.invites	Invite members to public channels
channels:write.topic	Set the description of public channels
chat:write	Send messages as @Incident Manager
commands	Add shortcuts and/or slash commands that people can use
groups:read	View basic information about private channels that "Incident Manager" has been added to
groups:write	Manage private channels that "Incident Manager" has been added to and create new ones
users:read	View people in a workspace

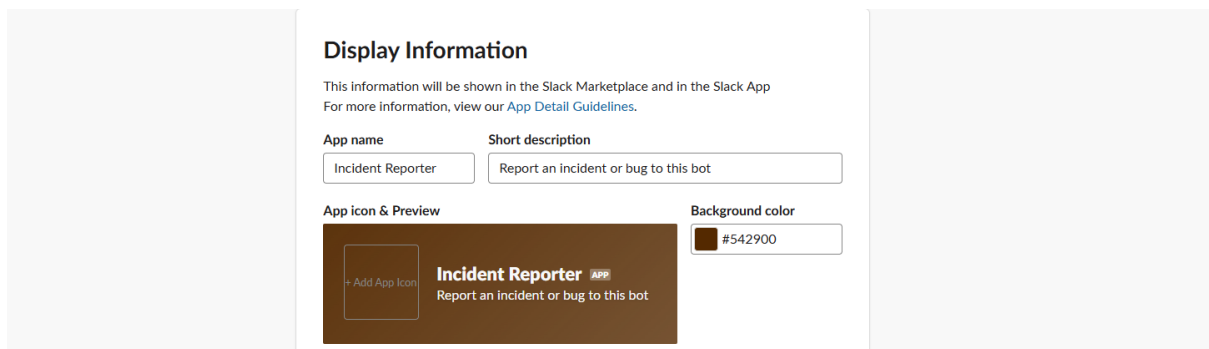
Add an OAuth Scope

Click on “Add an OAuth Scope” and add the following permissions to the manager bot:

```
channels:manage      commands
channels:read        groups:read
channels:write.invites groups:write
channels:write.topic  users:read
chat:write
```

And that’s it! You have successfully set up the manager bot as well. Now it’s important to know what information you should gather from your bot so that you may use it in your backend. We’ll do this in the next section.

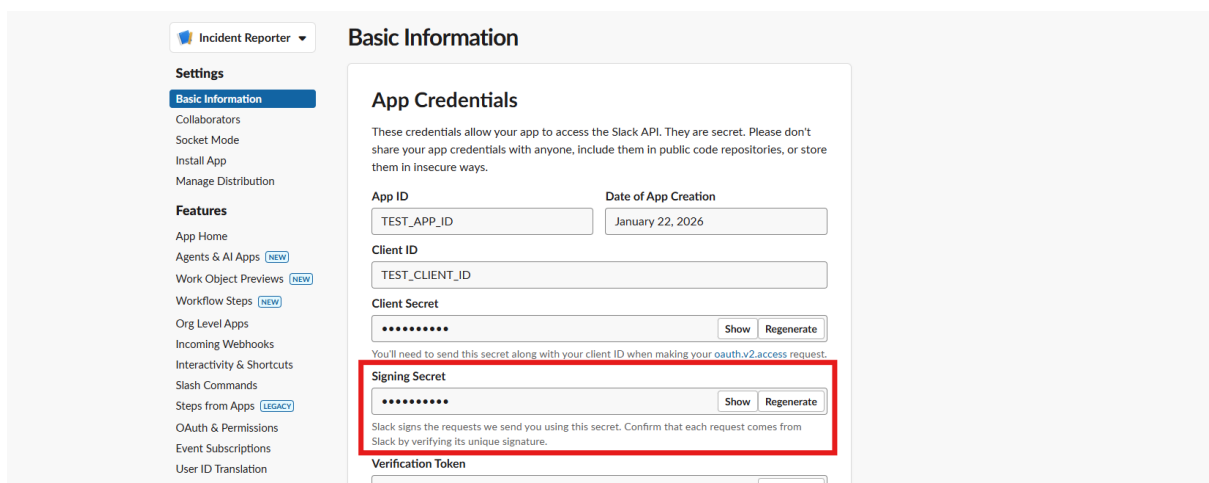
Of course, you may visit other sections of the bot settings and add some more configuration that might make it more production friendly. For example, in the “Basic Information” page you can find display information. You may tune them to your preference.



The screenshot shows the 'Display Information' section of a Slack app configuration. It includes a title 'Display Information', a note about where the information is shown, and a link to 'App Detail Guidelines'. Below this are two input fields: 'App name' (containing 'Incident Reporter') and 'Short description' (containing 'Report an incident or bug to this bot'). There is also an 'App icon & Preview' section showing a preview of the app icon and name, and a 'Background color' section with a color picker set to '#542900'.

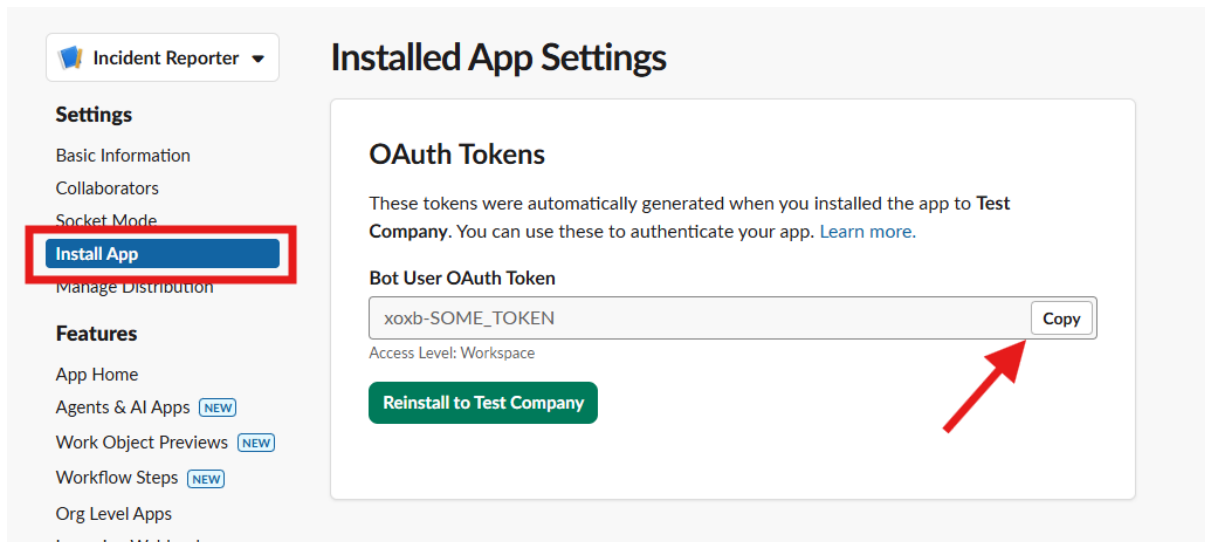
Environment Variables

Go back to your “Reporter” bot settings under “Basic Information” and copy the “Signing Secret” from “App Credentials”:



The screenshot shows the 'App Credentials' section of a Slack app configuration. It includes a title 'App Credentials', a warning about the security of these credentials, and several input fields: 'App ID' (containing 'TEST_APP_ID'), 'Date of App Creation' (containing 'January 22, 2026'), 'Client ID' (containing 'TEST_CLIENT_ID'), and 'Client Secret' (containing a masked value). Below these is a 'Signing Secret' section, which is highlighted with a red box. It contains a masked value and 'Show' and 'Regenerate' buttons. A note below the 'Signing Secret' states: 'You'll need to send this secret along with your client ID when making your oauth.v2.access request. Slack signs the requests we send you using this secret. Confirm that each request comes from Slack by verifying its unique signature.' There is also a 'Verification Token' section at the bottom.

Then go to the “Install App” section on the left side panel and copy the “Bot User OAuth Token”.



Paste these tokens to your “.env” file that you can use to test the backend app locally. **Remember that you have to do this for the manager bot too.** The variable in the “.env” file should be as per the following:

- **SLACK_SIGNING_SECRET_A** = {{SIGNING_SECRET_REPORTER_BOT}}
- **SLACK_BOT_TOKEN_A** = {{REPORTER_BOT_OAUTH_TOKEN}}
- **SLACK_SIGNING_SECRET_B** = {{SIGNING_SECRET_MANAGER_BOT}}
- **SLACK_BOT_TOKEN_B** = {{MANAGER_BOT_OAUTH_TOKEN}}

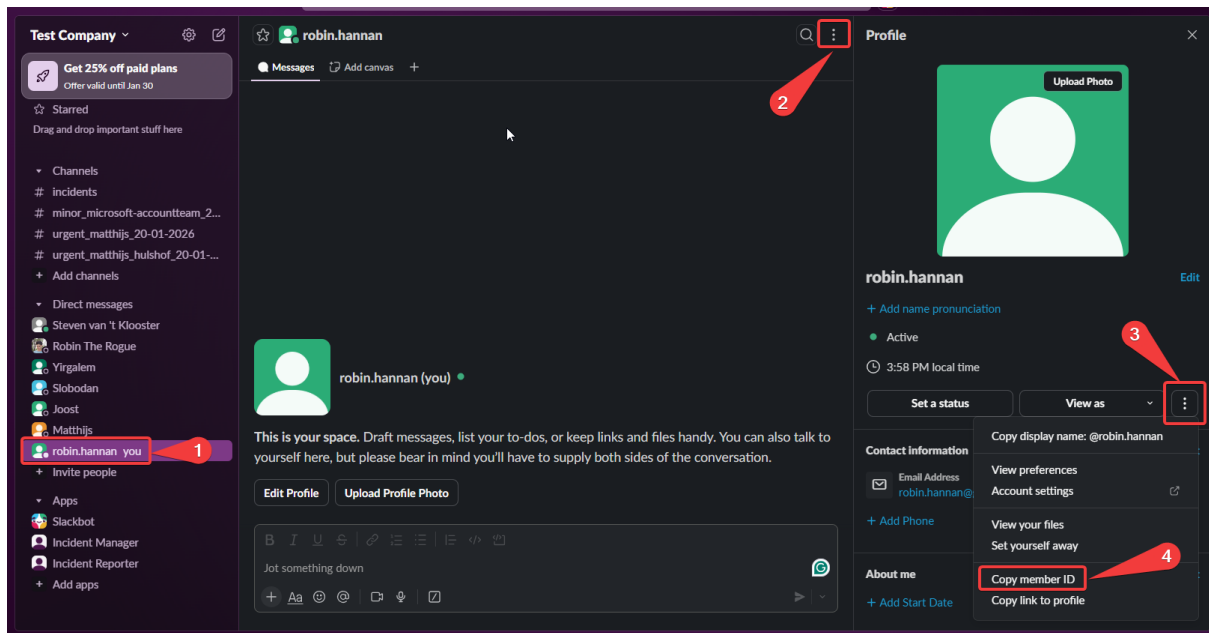
With this setup, you should be able to connect to your bots from the backend. You may even use a secure vault like Azure Key Vault in production. However, you might want to set up other environment variables from Slack that are also important for your app:

- **SLACK_DEVELOPER_USER_ID** = {{SLACK_ID_FOR_MAIN_DEVELOPER}}

This environment variable is set to whoever the main senior developer/head of engineering will be. With their Slack ID set, this user will be added to all new incoming incidents. They should also be workspace administrators. Essentially, choose a user who will be responsible for assigning the tasks to developers.

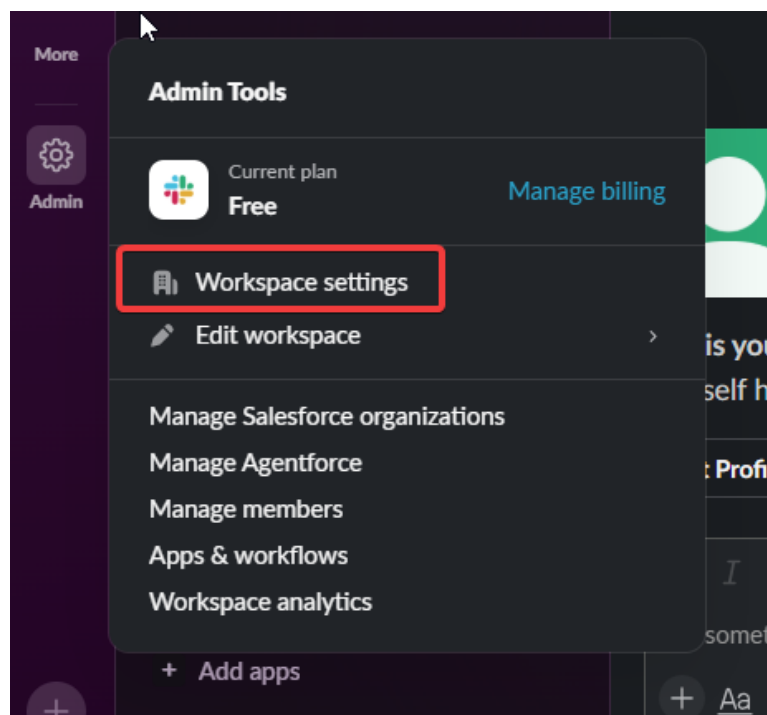
How to get Slack user ID?

Go to your Slack workspace and find a user. This will open up a private message window with them, on the top right side you have three dots, which will open up some menus. Click on “View full profile”, then click on the other three dots next to the “View as” button to “Copy member ID”. This is the Slack user ID.

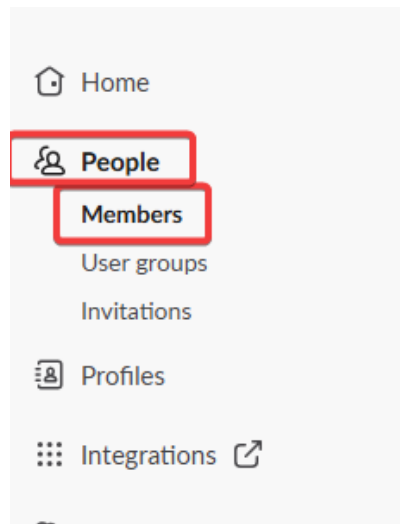


How to set a Slack user as Admin?

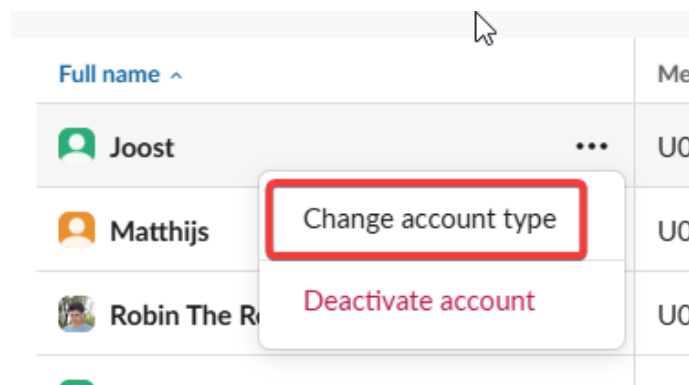
Your main developer in charge of assigning and viewing incidents has to be a workspace administrator. You can have multiple workspace administrators in one workspace.



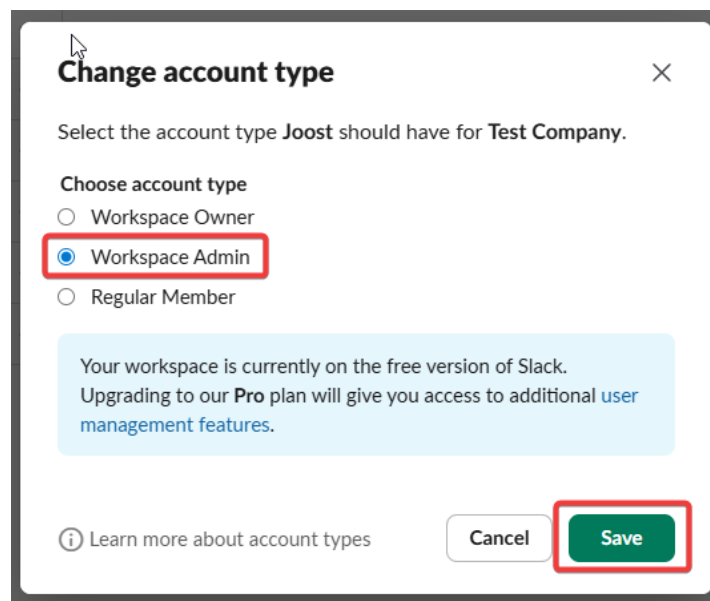
Ensure you are logged in as the main creator of the Slack workspace. On the left side of the workspace, there is a “Workspace settings” option. Click on that. This will open a new page.



On the left panel, you will find “Members” under “People”. Click on this option.



Next a list of members will show up who are in the workspace. Click on “Change account type” and select “Workspace Admin”.



Changes should come into effect immediately, and now that user can issue slash commands in any channel.